

Lab 2: Max Detection
Advanced Computerarchitecture

(B-KUL-JLI1QS)

Reinout Annaert

February 24, 2024



Session Date: 19/02/2024

Class: MElsw

Instructor: Alex Marinšek

This report will discuss the second assignment, addressing the guiding questions associated with it.

1 Correctness 1D approach

The three implementations correctly identify the maximum in the 1D array. Up to an array size of 2048, these identified maxima match for all three implementations because `syncthreads()` coordinates the communication between the threads of that same block, preventing any race conditions without the need for special adjustments.

2 1D approach

The three implementations for finding the maximum element in an array exhibit distinct time complexities as a function of input size. The CPU-based linear search has a linear time complexity ($O(n)$). However, it is essential to note that, due to parallelism, the linear growth of execution time for the GPU-based approach using atomic operations may exhibit a less pronounced linearity compared to the CPU-based linear search. Notably, the GPU-based search using reduction has a logarithmic time complexity ($O(\log_2 n)$), making it increasingly efficient as the input size grows. Despite the inherent differences in time complexity, it's interesting to observe that the GPU-based implementations, particularly the atomic and reduction approaches, demonstrate similar execution times. This phenomenon could be attributed to factors such as problem size, the level of parallelism achieved, and the efficiency of the algorithms employed.

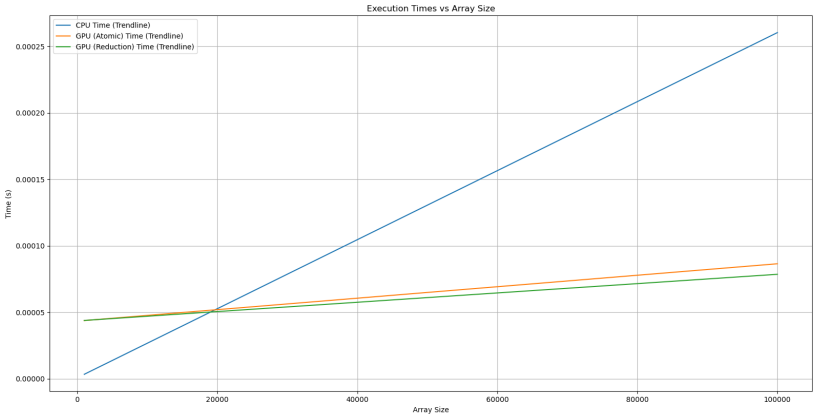


Figure 1: Execution Times in function of Array Length

3 Larger input sizes

The `syncthreads()` function in CUDA is designed to synchronize threads within a single block, ensuring that threads within the same block reach a common synchronization point before proceeding. This allows for effective collaboration and data sharing among threads within a

block. However, when dealing with array sizes beyond 2048, where multiple blocks are required, synchronization between blocks is not inherently handled by `syncthreads()`. Each block operates independently, and there is no automatic synchronization between them. This lack of inter-block synchronization can potentially lead to race conditions or inconsistencies among blocks.

The need to use multiple thread blocks arises when processing very large arrays to fully utilize all multiprocessors on the GPU. Each thread block is responsible for reducing a portion of the array. However, the challenge lies in how to communicate partial results between these thread blocks.

The idea of synchronizing across all thread blocks to easily reduce very large arrays is appealing, but CUDA lacks global synchronization. A possible solution could involve decomposing the problem into multiple kernels. A kernel launch serves as a global synchronization point. This approach avoids the need for global synchronization by decomposing the computation into multiple kernel invocations. For reduction operations, the code remains consistent across all levels, employing recursive kernel invocations to progress towards the final result.

4 Calling the kernel with much less threads than there are data elements

To address this, a practical strategy involves implementing a grid-strided loop. In this approach, each thread is assigned the responsibility of processing a consecutive subset of data elements. By introducing a loop inside the kernel, each thread iteratively processes its designated subset until all elements are covered. The loop adjusts the starting point and stride for each thread, ensuring comprehensive coverage of the data.

5 2D approach

`reduceBlockLevel` performs a reduction within each block, identifying the maximum value and its corresponding index. The results are stored in shared memory arrays (`blockVals` and `blockIndices`). The kernel is launched with a 2D grid and 1D block configuration to efficiently handle 2D data. Building upon the results of the first kernel, the `reduceFinal` kernel further reduces the block-level maximum values to find the global maximum for each row. The final results are stored in the output arrays (`index` and `result`). The `reduceGlobal` kernel operates globally, considering the entire 2D array. Similar to the second kernel, it identifies the maximum values and indices for each row, providing a benchmark for comparison.

For reference, a naive 2D CPU approach is implemented to find the global maximum using nested loops. The execution time is measured to compare the GPU-accelerated versions. An additional kernel, `atomicMaxKernel`, is implemented using atomic operations for maximum value computation. This serves as an alternative approach for comparison. The timing results obtained from the 2D GPU-accelerated approaches are akin to those depicted in Figure 1 for the 1D arrays.

The GitHub repository containing the source code can be found at *Github Repository*